AI - Assisted Coding-Assignment -16.3

HTNO:2303A52228
Batch:37

Lab 16 – Database Design and Queries: Schema Design and SQL Generation.

Task Description-1
Hospital Management Database.
Expected Output:
Normalized schema and SQL queries with joins.
Prompt:
Design a Hospital Management Database using SQL with Doctors, Patients, and Appointments tables.
Define primary keys, foreign keys, and constraints like NOT NULL and UNIQUE.
Establish relationships where doctors and patients are linked through appointments.
Write queries to list doctor appointments, retrieve patient history, and count patients per doctor using JOINs.
CODE AND OUTPUT :

hospital.sql ● MySQL Local.session.sql ●

```sql
CREATE DATABASE hospital_db;
USE hospital_db;
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    phone VARCHAR(15) UNIQUE );
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    age INT CHECK (age > 0),
    gender VARCHAR(10),
    phone VARCHAR(15) UNIQUE );
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    diagnosis VARCHAR(255),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id));
INSERT INTO Doctors (name, specialization, phone) VALUES
('Dr. Ravi', 'Cardiology', '1111111111'),
('Dr. Meena', 'Neurology', '2222222222');
INSERT INTO Patients (name, age, gender, phone) VALUES
('Abhi', 20, 'Male', '9999999999'),
('Rahul', 25, 'Male', '8888888888');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date, diag
(1, 1, '2025-03-10', 'Heart Checkup'),
(1, 2, '2025-03-11', 'BP Issue'),
```

List all appointments for a specific doctor.

hospital.sql ● MySQL Local.session.sql ● MySQL Local: multiple query results MySQL Local: SELECT d.name AS...

Detach file from MySQL Local | Run on active connection | Select block

```sql
SELECT d.name AS doctor_name, p.name AS patient_name,
 a.appointment_date, a.diagnosis
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.name = 'Dr. Ravi';
```

doctor_name	patient_name	appointment_date	diagnosis
Filter...	Filter...	Filter...	Filter...
Dr. Ravi	Abhi	2025-03-10	Heart Checkup
Dr. Ravi	Rahul	2025-03-11	BP Issue

Retrieve patient history by patient ID.

hospital.sql ● MySQL Local.session.sql ● MySQL Local: SELECT d.name AS... MySQL Local: SELECT p.name, a...

Detach file from MySQL Local | Run on active connection | Select block

```sql
SELECT p.name, a.appointment_date, a.diagnosis, d.name AS do
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;
```

name	appointment_date	diagnosis	doctor_name
Filter...	Filter...	Filter...	Filter...
Abhi	2025-03-10	Heart Checkup	Dr. Ravi
Abhi	2025-03-12	Headache	Dr. Meena

Count total patients treated by each doctor.

hospital.sql ● MySQL Local.session.sql ●

Detach file from MySQL Local | ▷Run on active connection | ≡ Select block

```sql
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS total_p
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

doctor_name	total_patients
Dr. Ravi	2
Dr. Meena	1

Explanation:
The Hospital Management Database is designed to store and manage information about doctors, patients, and their appointments in an organized way. Each doctor and patient has a unique ID, and appointments are used to connect them, ensuring proper relationships. Constraints like primary keys and foreign keys maintain data accuracy and avoid duplication. Using SQL queries with joins, we can easily retrieve doctor appointments, patient history, and analyze how many patients each doctor has treated.

Task Description-2
Real-Time Application: Online Shopping Database.
Expected Output:
Complete schema with relationships + SQL queries for analytics.
Prompt:
Design an Online Shopping Database using SQL with Users, Products, Orders, and OrderDetails tables.Define primary keys, foreign keys, and constraints to maintain data integrity and relationships.Write queries to retrieve all orders by a user, find the most purchased product, and calculate monthly revenue.
CODE AND OUTPUT :

ecommerce.sql > ...

Run on active connection | Select block

```sql
CREATE DATABASE ecommerce_db; USE ecommerce_db;
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL);
CREATE TABLE Products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL);
CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    order_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id));
CREATE TABLE OrderDetails (
    order_detail_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT CHECK (quantity > 0),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id));
INSERT INTO Users (name, email) VALUES
('Abhi', 'abhi@gmail.com'),('Rahul', 'rahul@gmail.com');
INSERT INTO Products (name, price) VALUES
('Laptop', 50000),('Phone', 20000);
INSERT INTO Orders (user_id, order_date) VALUES
(1, '2025-03-01'),(1, '2025-03-05'),(2, '2025-03-10');
INSERT INTO OrderDetails (order_id, product_id, quantity) VALUES
(1, 1, 1),(1, 2, 2),(2, 2, 1),(3, 1, 1);
```

Retrieve all orders by a user.

ecommerce.sql > ...

Run on active connection | Select block

```sql
SELECT u.name, o.order_id, o.order_date
FROM Users u
JOIN Orders o ON u.user_id = o.user_id
WHERE u.user_id = 1;
```

name	order_id	order_date
Filter...	Filter...	Filter...
Abhi	1	2025-03-01
Abhi	2	2025-03-05

Find the most purchased product.

ecommerce.sql > ...

Run on active connection | Select block

```sql
SELECT p.name, SUM(od.quantity) AS total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_sold DESC
LIMIT 1;
```

name	total_sold
Filter...	Filter...
Phone	3

Calculate total revenue in a given month.

ecommerce.sql > ...

Run on active connection | Select block

```sql
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE MONTH(o.order_date) = 3 AND YEAR(o.order_date) = 2025;
```

total_revenue
Filter...
160000.00

Explanation:
The Online Shopping Database manages users, products, and orders efficiently using a normalized schema. The Orders and OrderDetails tables establish relationships between users and products, avoiding redundancy. SQL queries using joins help retrieve user orders, identify the most purchased product, and calculate monthly revenue. Indexing and normalization improve performance and scalability of the system.

Task Description-3
Library Database.
Expected Output:
Schema + SQL queries demonstrating joins and conditions.
Prompt:
Design a Library Management Database using SQL with Books, Members, and Loans tables.Define primary keys, foreign keys, and constraints for data integrity.
Write queries to retrieve issued books, find overdue books, and count books per member using JOINs.Suggest indexing strategies to improve query performance.
CODE AND OUTPUT:

library.sql > ...

```sql
CREATE DATABASE library_db;
USE library_db;
CREATE TABLE Books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(150) NOT NULL,
    author VARCHAR(100),
    available_copies INT CHECK (available_copies >= 0));
CREATE TABLE Members (
    member_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE);
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY AUTO_INCREMENT,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id));
INSERT INTO Books (title, author, available_copies) VALUES
('DBMS', 'Korth', 5),
('Java', 'Herbert Schildt', 3);
INSERT INTO Members (name, email) VALUES
('Abhi', 'abhi@gmail.com'),
('Rahul', 'rahul@gmail.com');
INSERT INTO Loans (book_id, member_id, loan_date, return_date) VALUES
(1, 1, '2025-02-01', NULL),
(2, 2, '2025-01-01', NULL),
(1, 2, '2025-01-15', '2025-02-10');
```

Retrieve all books currently issued

library.sql > ...

Run on active connection | Select block

```sql
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

title	name	loan_date
Filter...	Filter...	Filter...
DBMS	Abhi	2025-02-01
Java	Rahul	2025-01-01

Find overdue books (loan > 30 days)

library.sql > ...

Run on active connection | Select block

```sql
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND DATEDIFF(CURDATE(), l.loan_date) > 30;
```

title	name	loan_date
Filter...	Filter...	Filter...
DBMS	Abhi	2025-02-01
Java	Rahul	2025-01-01

Count books loaned by each member

library.sql > ...

Run on active connection | Select block

```sql
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id;
```

name	total_books
Abhi	1
Rahul	2

Explanation:
The Library Management System stores information about books, members, and loans in a structured way. The Loans table connects books and members, maintaining relationships using foreign keys. Queries with joins help retrieve issued books, identify overdue loans, and count books borrowed by each member. Indexing improves query performance, especially for large datasets.

Task Description-4
Optimize Queries.
Expected Output:
Both queries return the same result: all books written by UK authors.
Optimized query performs faster for larger datasets.
Prompt:
Analyze and optimize an SQL query that retrieves books written by UK authors. Replace the subquery with a JOIN to improve performance. Ensure both original and optimized queries return the same result. Suggest indexing techniques to enhance query efficiency for large datasets.
CODE AND OUTPUT :
Original query

library.sql > ...

Run on active connection | Select block

```sql
SELECT *
FROM Books
WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'
);
```

book_id	title	author	available_copies
3	SQL Book	NULL	NULL

Optimized query

library.sql > ...

Run on active connection | Select block

```sql
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

book_id	title	author	available_copies	author_id
3	SQL Book	NULL	NULL	3

Explanation:
The original query uses a subquery to find books written by UK authors, which can be slower because it runs an inner query separately. The optimized query replaces the subquery with a JOIN, directly linking the Books and Authors tables. This improves performance and reduces execution time, especially for large datasets. Both queries produce the same result, but the JOIN-based query is more efficient and scalable.

Task Description-5
University Course Registration.
Prompt:
The University Course Registration system manages students, courses, and faculty efficiently. The Registrations table connects students and courses, allowing many-to-many relationships. SQL queries using joins help retrieve enrolled students, identify faculty teaching multiple courses, and find popular courses based on registrations.

CODE AND OUTPUT :

library.sql > ...

Run on active connection | Select block

```sql
CREATE DATABASE university_db;
USE university_db;
CREATE TABLE Students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    faculty_id INT);
CREATE TABLE Faculty (
    faculty_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(100));
CREATE TABLE Registrations (
    reg_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id));
INSERT INTO Faculty (name, department) VALUES
('Dr. Rao', 'CSE'),
('Dr. Meena', 'ECE');
INSERT INTO Courses (course_name, faculty_id) VALUES
('DBMS', 1),
('Java', 1),
('Networks', 1),
('Signals', 2);
```

List all students enrolled in a specific course.

library.sql > ...

Run on active connection | Select block

```sql
SELECT s.name, c.course_name
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
JOIN Courses c ON r.course_id = c.course_id
WHERE c.course_name = 'DBMS';
```

name	course_name
Filter...	Filter...
Abhi	DBMS
Rahul	DBMS

Find faculty members teaching more than 2 courses.

library.sql > ...

Run on active connection | Select block

```sql
SELECT f.name, COUNT(c.course_id) AS total_courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;
```

name	total_courses
Filter...	Filter...
Dr. Rao	3

Retrieve courses with the highest number of registrations.

library.sql > ...

Run on active connection | Select block

```sql
SELECT c.course_name, COUNT(r.student_id) AS total_students
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY total_students DESC
LIMIT 1;
```

course_name	total_students
Filter...	Filter...
DBMS	2

Explanation:
AI was used to add comments and docstrings to each API endpoint created using FastAPI. These docstrings describe the purpose of each endpoint, the request method used, and the type of data expected. When the API runs using Uvicorn, FastAPI automatically generates interactive documentation through Swagger.